

Теория по linux

Владислав Богданов

2026-06-24

Оглавление

1. Полезные утилиты	1
2. Инструменты	1
3. makefile	1
4. Компилятор	2
4.1. Ключи	2
5. Модель КИС	3
6. Окружение	3
6.1. Библиотеки	3
6.2. Команды	3
7. Механизмы ввода-вывода	3
8. Библиотеки	4
9. Файловые системы	5
9.1. Библиотека	5
9.2. Команды	5
9.2.1. Разреженные файлы (sparse files)	5
9.2.2. Функции контроля	6
9.2.3. Символические ссылки	6
10. Справка по командам	7
11. IPC Межпроцессное взаимодействие	7
11.1. Pipe	7
11.1.1. Неименованный канал	7
11.1.2. pipe() + fork()	8
11.2. Именованный канал	8
12. Очередь сообщений	9
13. Shared Memory	12
13.1. Почему не "разделяемая память"?	12
13.2. Можно ли обратиться к чужой памяти?	12
13.3. Виды памяти (с точки зрения процесса)	12
13.4. Как работает Shared Memory?	12
13.5. Проблема синхронизации	13
13.6. Семафоры: механизм синхронизации	13
14. Сокеты	14
14.1. Введение	14
14.2. Пример 1: Файловые сокеты (Unix)	15
14.3. Пример 2: Парные сокеты (socketpair)	15
14.4. Пример 3: Сетевые сокеты (INET)	15
14.5. Практические детали	16
14.5.1. Управление временем ожидания	16

14.5.2. Порт 33333	16
14.5.3. Протокол передачи данных	16
14.5.4. Взаимодействие с Telnet	17
14.5.5. Особенности очистки	17
15. Сигналы	17
15.1. Ключевые особенности сигналов:	17
15.1.1. Сигнал может быть:	17
15.1.2. Игнорирование и блокировка сигналов	17
15.1.3. Популярные сигналы	18
15.2. Работа с сигналами в С	18
15.2.1. Устаревшие и современные подходы	18
15.2.2. Практические примеры	18
15.3. Рассылка и генерация сигналов	18
16. Потоки	19
17. Демоны	19
17.1. Введение	19
18. Книги	19

1. Полезные утилиты

- `nm` - позволяет посмотреть функции и тп. в бинарном файле, стоит смотреть флаги `T`-что то типа внутренних функций, `U`-сторонние функции
- `less` - просмотр файлов в командной строке постранично
- `objdump` -
- `time` - измерить время
- `du` - реальный размер файла

2. Инструменты

- `make`
- `ar`
- `gcc`

3. makefile

Включает в себя:

- Цель
- Требование
- Команда (перед команой ставится табуляция)

Комментарии обозначаются через `#`

```
# комментарий
#с подключением статической библиотеки
#hello example makefile

hello: main.o libhello.a
    gcc -o hello main.o -L. -lhello

main.o: main.c
    gcc main.c -c

libhello.a : hello.o goodbye.o
    ar cr libhello.a hello.o goodbye.o

hello.o: ../library/static/hello.c
    gcc -I../library/static -c ../library/static/hello.c

goodbye.o: ../library/static/goodbye.c
    gcc -I../library/static -c ../library/static/goodbye.c
```

```
clean:
    rm -f *.o *a hello
```

```
# комментарий
#с подключением динамической библиотеки

#hello example makefile

hello: main.o libhello.so
    gcc -o hello main.o -L. -lhello -Wl,-rpath,.
#    rm -f *.o

main.o: main.c
    gcc main.c -c

libhello.so : hello.o goodbye.o
    gcc -shared -o libhello.so hello.o goodbye.o

hello.o: ../library/static/hello.c
    gcc -I../library/static -c ../library/static/hello.c -fPIC

goodbye.o: ../library/static/goodbye.c
    gcc -I../library/static -c ../library/static/goodbye.c -fPIC

clean:
    rm -f *.o *so hello
```

4. Компилятор

4.1. Ключи

- **O** - оптимизация по умолчанию O1
- **-c** - создает объектный файл без линковки
- **l** - линковка с библиотекой к примеру math.h -lm
- **-L** - это указывает что искать в директории, **.** - текущая
- **-fPIC** - position independent code, позиционно независимый код, данная опция сообщает компилятору, адреса должны быть плавающие, не абсолютные.
- **-Wl, -rpath, .** - говорит откуда должна запускаться динамическая библиотека.
 - **-rpath, .** - из текущего каталога === Команды
- **strip** - вычищает из исполняемого файла лишние ключи и команды, используется в релизе

5. Модель КИС

Клиент - Интерфейс - Сервер

"main" - stdio.h - stdio

6. Окружение

Окружение - набор пар (переменная = значение), которые доступны каждому пользовательскому процессу. Название `environ`.

При запуске родительский процесс передаёт копию окружения дочернему, который может делать со своей копией, все что угодно. Это не повлияет на работу системы в целом. В линукс есть несколько способов взаимодействия с переменными окружения из процессов.

Есть библиотека `unistd.h` все необходимые базовые объявления для работы в линукс. Внутри этой библиотеки есть массив `environ` который хранит построчно список переменных и значений.

```
#вывод окружения
$env
```

6.1. Библиотеки

- `unistd.h`
- `errno.h`
- `fctrl.h`

6.2. Команды

- `abort()` - аварийное завершение команды
- `exit(0)` - мягкое завершение команды из любой точки
- `getenv(name)`
- `setenv(name*, value* flag(overwrite 0-nowrite, !=0 -update))`
- `unsetenv` - удаляет переменную окружения из среды текущего процесса

7. Механизмы ввода-вывода

Все есть потоковый ввод-вывод (низкоуровневый ввод - вывод)

`syscall` - системный вызов (`unistd.h`)

Примеры:

- open()
- close()
- read()
- write()
- lseek() - переместить курсор

Работа с open()

Пример:

```
int fd = open("file.txt", O_WRONLY | O_CREAT | O_EXCL, S_IRUSR | S_IWUSR);
O_CREAT – создать файл;
O_EXCL – не перезаписывать, если файл уже существует;
S_IRUSR | S_IWUSR – права доступа (чтение/запись для владельца).
open() возвращает дескриптор или -1 в случае ошибки.
```

Файловый дескриптор FILE- высокоуровневая абстракция все механизмы которого не доступны при низкоуровневых операциях, на низком уровне используют целочисленные файловые дескрипторы(int) и про FILE можно забыть. Их номера скрыты в ядре кроме потоков ввода вывода их номерация соответствует.

Нужно напечатать столбцом и получится таблица

io

012

0 - input, 1 - output, 2 - поток ошибок

У каждого процесса свои дескрипторы.

```
$ ls -al /dev | grep std
#максимальное количество дескрипторов
$ ulimit -n
```

8. Библиотеки

Библиотеки в Linux должны начинаться со слова lib, это связано с тем как они потом компилируются.

- статические
 - расширение.a
 - a - archive

- автономны так как идут в комплекте с программой
- быстрее
- динамические
 - расширение .so
 - so - shared object
 - в программе ссылка на библиотеки
 - экономит ресурсы
 - не всегда требуют пересборки использующей программы

9. Файловые системы

Все есть файл

Существует несколько типов файловых систем: * Обычные (пользовательские) — содержат файлы, директории, документы. * Служебные — обеспечивают взаимодействие с ядром, устройствами и процессами. Среди них:

- /dev - (устройства);
- /sys - (sysfs — информация о устройствах и драйверах);
- /proc - служебное хранилище информации о процессах, файлы хранящиеся в директории не настоящие файлы, их создает linux на основе данных из оперативной памяти, служебных данных процессов, самого ядра, модулей ядра и др.

9.1. Библиотека

- dirent.h
- fcntl.h

9.2. Команды

- scandir
- alphasort
- opendir
- readdir
- closedir

9.2.1. Разреженные файлы (sparse files)

Пример

Создается файл.
Записывается строка.

Смещаемся на 17 МБ и пишем еще раз.

Итог: логический размер файла – 17 МБ, физический (по `du`) – 8 байт.

Почему?

Linux не выделяет физическое место на диске для незаполненных областей, содержащих только нули. Это позволяет экономить пространство.

Где применимо:

Сетевые терминальные системы с квотами. Передача "разреженных" потоков между процессами.

9.2.2. Функции контроля

- `fcntl`
- `ioctl`

Блокировка файлов: `fcntl`

Пример `testlogs`:

Используется функция `fcntl` с флагом `F_SETLK` для установки блокировки. Блокировка не "жесткая": работает только, если другой процесс сам проверяет её. Объект блокировки – диапазон байт (указывается `l_start`, `l_len`, `l_type`, `l_whence`). Возможные значения `l_type`:
`F_WRLCK` – блокировка на запись.
`F_RDLCK` – блокировка на чтение.
`F_UNLCK` – снятие блокировки.

9.2.3. Символические ссылки

Примеры:

`/proc/self/exe`, `/proc/self/cwd` – символические ссылки.
Используются для доступа к текущему исполняемому файлу и рабочей директории.
Создание: `ln -s source linkname`

В линукс 7 типов файлов:

- обычные файлы, regular files (`touch ...`)
- директории, directory (`mkdir ...`)
- символические ссылки, symbolic links (`ln ...`)
- символьные устройства, character device (`mknod ...`)
- блочные устройства, block device (`mknod ...`)

- файловые каналы, pipe (mkfifo)
- сокеты, socket !

Тип можно узнать по первому символу в ls -l.

10. Справка по командам

```
vlad@BogdanovVV:~/Рабочий стол$ man -f open
vlad@BogdanovVV:~/Рабочий стол$ man open
vlad@BogdanovVV:~/Рабочий стол$ man 2 open
```

11. IPC Межпроцессное взаимодействие

Инструменты линукс обеспечивающее передачу информации между процессами операционной системы.

Проблемы монолита

- долгая загрузка
- сложность изменения
- влияние ошибок более серьезно
- тяжело передавать и выводить информацию

11.1. Pipe

11.1.1. Неименованный канал

Самый распространенный вариант внутри программной передачи данных. Программа запускает из себя другую программу как процесс и следит за тем что она ей передаст. При этом мы можем использовать вывод другой программы вообще не вдаваясь как она создана. За работу неименованного канала отвечает `open()` `pclose()`. При этом при использовании неименованного канала мы передаем информацию в одну сторону.

Если нужна передача в обратную сторону то нужна другая концепция `fork()`. При этом применяются два канала связи, это будут файлы, это будут потоки данных в оперативной памяти. Где один процесс читает, а другой пишет. `pipe()` - создает два неименованных однонаправленных канала. У них два дескриптора.

Особенности:

- Создаются во время исполнения программы.
- Существуют в оперативной памяти.
- Не имеют имени в файловой системе.

- Однонаправленные: один процесс пишет, второй — читает.
- Используются через вызовы `pipe()`, `popen()`, `fork()`, `read()`, `write()`

Пример: `popen()` и `fread` Функция `popen()` запускает внешнюю команду и открывает поток, по которому можно получить ее вывод. Это упрощает чтение результата других программ.

```
FILE* f = popen("ls", "r");
fread(...);
pclose(f);
```

Недостаток `popen()` — она запускает команду через `shell`, что может быть неэффективно и небезопасно.

11.1.2. `pipe()` + `fork()`

Функция `pipe(int fd[2])` создает два файловых дескриптора: `fd[0]` — для чтения; `fd[1]` — для записи. Далее вызывается `fork()`, который создает копию процесса. После этого каждый из процессов (родительский и дочерний) может использовать пайп для обмена данными.

```
int pipefd[2];
pipe(pipefd);
pid_t pid = fork();

if (pid == 0) {
    // дочерний процесс: читает
    close(pipefd[1]);
    read(pipefd[0], ...);
} else {
    // родительский процесс: пишет
    close(pipefd[0]);
    write(pipefd[1], ...);
}
```

11.2. Именованный канал

Когда нужно, чтобы разные программы (а не процессы одной программы) взаимодействовали между собой, используют именованные каналы — FIFO-файлы.

`mkfifo()` Создает файл специального типа в файловой системе:

```
mkfifo("myfifo", 0666);
```

Далее обычные вызовы `fopen`, `fprintf`, `fgets` позволяют писать и читать данные. Важно: `unlink("myfifo")` удаляет канал после использования, но только если в нем нет данных.

Особенности реализации

- Неименованные каналы — это временные, процессо-локальные средства связи.
- Именованные каналы — постоянные, видимые в файловой системе.
- `popen()` удобно использовать для чтения вывода утилит.
- `execve()` — для замены процесса другой программой.
- `fork()` — создает новые процессы.
- `pipe()` — соединяет процессы очередью (FIFO).
- `dup2()` — перенаправляет стандартные потоки.

Ключевые функции

Функция	Назначение
<code>popen()</code>	Открыть поток к внешней программе
<code>pipe()</code>	Создать неименованный канал
<code>fork()</code>	Создать дочерний процесс
<code>execve()</code>	Заменить процесс другим
<code>mkfifo()</code>	Создать именованный канал (FIFO)
<code>read()</code>	Считать данные из дескриптора
<code>write()</code>	Записать данные в дескриптор
<code>dup2()</code>	Перенаправить один поток в другой
<code>unlink()</code>	Удалить файл или канал

12. Очередь сообщений

Очереди сообщений — один из механизмов межпроцессного взаимодействия (IPC) в Linux наряду с пайпами (`pipe`) и разделяемой памятью (`shared memory`). В отличие от пайпов, которые появились с самого начала существования Unix, очереди сообщений были введены позже, как часть System V IPC, чтобы обеспечить: удобную и системно-управляемую передачу данных между процессами; поддержку обмена структурированной информацией (а не просто потоками байтов); произвольный доступ к сообщениям внутри очереди (в отличие от строго FIFO-пайпов); идентификацию сообщений и возможность приоритизации.

Как работают очереди сообщений?

Общие принципы:

- Очередь создается в системе и управляется ядром ОС.
- Для доступа к очереди используется уникальный ключ (`key_t`), который известен всем взаимодействующим процессам. Ключ может быть сгенерирован вручную (например, 1174) или автоматически через вызов `ftok(path, id)`, основанный на пути к исполняемому файлу.

- Каждое сообщение имеет обязательное поле `long mtype`, определяющее его тип/приоритет.
- Структура сообщения может быть произвольной (любые данные), но обязательно начинается с `long`.

Основные системные вызовы

Вызов	Назначение
<code>msgget()</code>	Получение или создание очереди
<code>msgsnd()</code>	Отправка сообщения в очередь
<code>msgrcv()</code>	Чтение сообщения из очереди
<code>msgctl()</code>	Управляющие операции: удаление очереди, получение статуса и др.

Особенности работы с сообщениями

1. Извлечение сообщений по типу

Поле `mtype` можно использовать для выбора сообщений:

- больше 0 — получить первое сообщение указанного типа;
- равно 0 — получить первое сообщение любого типа (FIFO);
- меньше 0 — получить сообщение с $mtype \leftarrow \text{abs}(mtype)$. Это позволяет реализовать приоритизацию, обработку системных и пользовательских событий, контроль очередности и т.п.

2. Передача данных

- Сообщение — это последовательность байтов, которую передает один процесс другому.
- Если структура неизвестна, сообщение интерпретировать невозможно.
- Требуется общая библиотека структур на клиенте и сервере (например, `Request`, `Response`).
- Возможны проблемы выравнивания памяти — поля должны быть кратны 4 байтам.
- Для передачи используют буферы размером страницы памяти (обычно 4 КБ).
- Для проверки целостности можно использовать контрольные суммы и пакеты (`buffer + checksum`).

Свойства очередей сообщений

- Сообщения хранятся в очереди до чтения. Поддерживается накопление.
- Очередь должна быть удалена владельцем после использования — иначе произойдет утечка ресурсов.
- Произвольная структура сообщений, но максимальный размер:
 - ~8 КБ на 32-битных системах;

- ~16K сообщений на очередь (зависит от ядра).

Сравнение с другими IPC-инструментами

Механизм	Направление	Буферизация	Передача структур	Очередность	Произвольный доступ
Pipe	Однонаправленный	Есть	Нет (байты)	FIFO	Нет
Shared Memory	Двунаправленный	Нет	Да	Нет	Да (но вручную)
Message Queue	Двунаправленный	Есть	Да (структуры)	FIFO/по mtype	Да

Архитектурные замечания

- Для полноценного отклика на события (например, из GUI), приложения реализуют цикл событий (event loop).
- Обработчики событий (handlers) должны быть реализованы как функции или методы классов.
- Важно избегать блокировок в этих обработчиках — иначе ОС решит, что программа не отвечает.

Пример: простой чат на message queue

- Сервер и клиент обмениваются структурированными сообщениями через одну очередь.
- Используются типы сообщений: 1 — от клиента, 2 — от сервера.
- Структура:

```
struct message {
    long mtype;
    pid_t sender_pid;
    char text[256];
};
```

Сценарий:

- Клиент отправляет сообщение → Сервер читает → отправляет ОК в ответ.
- Возможна реализация более сложных схем: "радио"-чат, приоритизация и т.п.

Практические советы

- Никогда не забывайте вызывать msgctl(..., IPC_RMID, ...) — иначе очередь зависнет.
- Размер сообщения всегда рассчитывайте как sizeof(msg) - sizeof(long) — т.к. mtype обрабатывается системой отдельно.
- В качестве key_t предпочтительнее использовать ftok(), а не вручную выбранные числа.

- Используйте `IPC_CREAT` | `IPC_EXCL`, чтобы избежать конфликта при создании уже существующей очереди.
- Следите за буферизацией — возможны тайминговые баги, особенно в системах реального времени.

13. Shared Memory

Shared Memory — это механизм взаимодействия между процессами (IPC), позволяющий нескольким процессам работать с одним и тем же участком оперативной памяти. Это один из самых быстрых способов обмена данными между процессами, так как не требует копирования данных из одного адресного пространства в другое.

13.1. Почему не "разделяемая память"?

В русской литературе часто используется термин "разделяемая память", но он может вводить в заблуждение: "разделенная" память может означать совершенно другое (например, NUMA-архитектуры, где у каждого процессора свой блок ОЗУ). Поэтому английский термин `shared memory` — более точный и нейтральный.

13.2. Можно ли обратиться к чужой памяти?

- Нет. В нормальных условиях процесс не может напрямую обращаться к памяти другого процесса.
- У каждого процесса свое адресное пространство.
- Любая попытка обратиться к "чужой" памяти считается нарушением и расценивается как хак или эксплойт.

13.3. Виды памяти (с точки зрения процесса)

- Своя память — инструкции и переменные текущего процесса.
- Чужая память — все, что принадлежит другим процессам или ядру.
- Куча (`heap`) — динамически выделяемая память, которую процесс сам запрашивает и освобождает.

13.4. Как работает Shared Memory?

1. Память выделяется при помощи системного вызова `shmget`.
2. Отображается в адресное пространство процесса вызовом `shmat`.
3. Для корректной работы требуется согласованность действий между процессами (например, кто и когда читает или пишет).

Пример: два процесса и переменная `x`

Допустим, `x = 0`. Два процесса одновременно выполняют `x++`. Возможные значения:

- Идеально: $x = 2$.
- На практике: может быть 1 или даже 0.

Почему?

- $x++$ не атомарная операция.
- Процессы могут "переписывать" друг друга.

13.5. Проблема синхронизации

При одновременной работе с памятью:

- **Чтение + запись:** возможны "битые" данные.
- **Запись + запись:** конфликт, потеря данных.
- **Чтение + чтение:** безопасно.

Вывод: **необходим механизм синхронизации.**

13.6. Семафоры: механизм синхронизации

Зачем нужны?

Семафоры — это **флаги или счетчики**, которые контролируют доступ к ресурсу.

Пример с железной дорогой:

- Переезд = общая память.
- Поезда = процессы.
- Семафор = механизм, который разрешает/запрещает проезд.

Как работает?

1. Подъехал — **проверил семафор.**
2. Если открыт — **пытаемся закрыть в свою пользу.**
3. После проезда — **открываем обратно.** Если не удалось закрыть — **ждем.**

Спинлоки — плохая идея

- `while (flag != 1) {}` — нагружает процессор.
- Постоянный опрос = **высокая нагрузка на CPU.**
- Используется только в ядре ОС или в крайних случаях. Вместо этого — **семафоры**, которыми управляет ОС.

API работы с shared memory и семафорами

Shared Memory


```
key_t key = ftok("filename", 1); // Уникальный ключ
int shmid = shmget(key, size, IPC_CREAT | 0666); // Создать память
void* ptr = shmat(shmid, NULL, 0); // Присоединить
shmdt(ptr); // Отсоединить
shmctl(shmid, IPC_RMID, NULL); // Удалить
```

Семафоры

```
int semid = semget(key, 2, IPC_CREAT | 0666); // Создание 2 семафоров
semctl(semid, 0, SETVAL, 1); // Инициализация
semop(semid, &operation, 1); // Выполнение операции
```

Операции semop

- `sem_op < 0` — запросить ресурс.
- `sem_op > 0` — освободить ресурс.
- `sem_op = 0` — ждать, пока значение станет 0.

Пример флажков:

- `serverLock` — сервер блокирует ресурс.
- `clientLock` — клиент блокирует ресурс.
- `turn` — чей ход.
- `readLast` — кто последний работал.

14. Сокеты

14.1. Введение

Сокет — это программный интерфейс, обеспечивающий обмен данными между процессами. В Linux сокеты, как и все остальное, — это файлы. Это значит, что вы взаимодействуете с ними через файловые дескрипторы, используя функции `read`, `write`, `close` и др. Сокеты часто ассоциируются с сетевыми соединениями, но на самом деле они универсальны: их можно использовать и локально.

Типы сокетов Сокеты классифицируются по: **Семейству (domain):**

- `AF_UNIX` — локальные файловые сокеты.
- `AF_INET`, `AF_INET6` — сетевые сокеты.
- `AF_PACKET`, `AF_IPX` и др. — специализированные типы.

Типу передачи (type):

- `SOCK_STREAM` — потоковые (надежная передача, аналог TCP).

- SOCK_DGRAM — дейтаграммные (ненадежная передача, аналог UDP).
- SOCK_RAW, SOCK_SEQPACKET — менее распространенные.

14.2. Пример 1: Файловые сокеты (Unix)

Работа через файловые сокеты — аналог использования именованных каналов (FIFO), но с более мощным интерфейсом.

Создание сокета:

```
socket(AF_UNIX, SOCK_DGRAM, 0)
```

Привязка к файлу:

```
bind(sockfd, (struct sockaddr *)&addr, addrlen)
```

Передача и прием:

```
sendto(), recvfrom() — для отправки и получения данных по адресу.
```

Важно: такие сокеты работают только на одной машине. Даже если файл доступен по сети, соединения не получится.

14.3. Пример 2: Парные сокеты (socketpair)

Функция **socketpair()** позволяет создать два связанных сокета в памяти. Это альтернатива неименованным каналам (pipe()).

- Работает только в пределах одного процесса (или между процессом и его потомком после fork()).
- Используются для организации двустороннего обмена.
- Пример:

```
int sv[2];  
socketpair(AF_UNIX, SOCK_STREAM, 0, sv);
```

Мнение лектора В большинстве случаев неименованные каналы проще и эффективнее. Парные сокеты применяются реже и, по сути, избыточны.

14.4. Пример 3: Сетевые сокеты (INET)

Самый широко применяемый тип сокетов. Используется для обмена данными между компьютерами по сети.

Создание:

`socket(AF_INET, SOCK_STREAM, 0)` – TCP

`socket(AF_INET, SOCK_DGRAM, 0)` – UDP

Привязка к порту:

`bind(sockfd, (struct sockaddr_in *)&addr, sizeof(addr))`

Прослушка соединений:

`listen(sockfd, backlog)`

Принятие соединения:

`accept(sockfd, ...)`

Подключение клиента:

`connect(sockfd, ...)`

Отправка и прием:

`read()`, `write()` или `recv()`, `send()`

Особенности:

Сервер всегда использует `bind()`, `listen()`, `accept()`.

Клиент – `connect()`.

14.5. Практические детали

14.5.1. Управление временем ожидания

Сокеты по умолчанию блокируют выполнение:

- `read()` ждет, пока появятся данные.
- `accept()` ждет подключений.

Для управления этим используются: * Флаг `O_NONBLOCK` * Системный вызов `select()` с таймаутами (`struct timeval`)

14.5.2. Порт 33333

В демонстрационном примере использовался учебный порт 33333, который часто бывает свободен.

14.5.3. Протокол передачи данных

Чтобы понять, когда данные закончились, важно договориться о формате передаваемой информации:

- Определенный размер пакета.
- Завершающий символ (`\n`, `\0`).

- Контрольная сумма (CRC).
- Фиксированный формат (заголовок, тело, хвост).

Иначе вы не сможете различить просто паузу и конец сообщения.

14.5.4. Взаимодействие с Telnet

Лектор показал, как подключаться к серверу через telnet: * telnet 127.0.0.1 33333 — простой способ протестировать работу сервера. * Отправка сообщений в виде строк.

14.5.5. Особенности очистки

После завершения работы сервера:

- Порт остается занятым несколько секунд.
- Чтобы порт освобождался сразу, необходимо использовать флаг SO_REUSEADDR при создании сокета (через setsockopt()).

15. Сигналы

Сигналы в Linux — это механизм межпроцессного взаимодействия, суть которого сводится к тому, что ОС передаёт определённые короткие сообщения (сигналы) процессу.

15.1. Ключевые особенности сигналов:

Обработка сигнала происходит либо немедленно, либо не происходит вообще (если игнорируется);

15.1.1. Сигнал может быть:

- Обработан (через функцию-обработчик);
- Игнорирован;
- Обработан по умолчанию ОС (например, завершение процесса);
- При этом выполнение кода может продолжаться с места, на котором было прервано.

15.1.2. Игнорирование и блокировка сигналов

Игнорирование — сигнал уничтожается, программа его «не замечает»;

Блокировка — сигнал сохраняется, но не обрабатывается до снятия блокировки. Сигналы можно блокировать через системные вызовы. При этом можно узнать, что сигнал был получен, но реакция произойдёт только при разблокировке.

Некоторые сигналы нельзя игнорировать или блокировать:

- **SIGKILL** (9) — аварийное завершение процесса;

- **SIGSTOP** (19) — остановка процесса.

15.1.3. Популярные сигналы

- **SIGINT** — Ctrl+C в терминале, прерывает выполнение;
- **SIGHUP** (1) — «повесить трубку», сигнал перезапуска/перечитывания конфигурации;
- **SIGCHLD** — завершение дочернего процесса;
- **SIGUSR1** и **SIGUSR2** — пользовательские сигналы (номера ~10–12), могут использоваться для кастомных реакций.

Использовать пользовательские сигналы для бизнес-логики не рекомендуется.

15.2. Работа с сигналами в С

Мы можем:

- Назначить обработчик с помощью `sigaction()`;
- Заблокировать/разблокировать сигнал с помощью `sigprocmask()`;
- Использовать `sigwait()`; для «усыпления» до прихода сигнала.

Важно: обработчики должны быть реентерабельны — не использовать глобальные переменные без синхронизации. `printf()` в обработчике — плохая идея.

15.2.1. Устаревшие и современные подходы

Раньше использовался массив `sys_siglist` для описаний сигналов. Сейчас — `strsignal()` (функция); Примеры адаптированы под новые стандарты (2024+); Linux активно обновляется, старые конструкции могут не работать.

15.2.2. Практические примеры

Программа SIGDEMO:

- Обработывает сигнал **SIGTERM** через `TermHandler()` — выводит сообщение и завершает работу;
- Блокирует сигнал **SIGHUP**, который не вызывает реакции без снятия блокировки.

Программа с `sigwait()`:

- Блокирует сигнал **SIGHUP**, а затем «ждёт» его через `sigwait()` — нагрузка на CPU отсутствует, программа просыпается только при получении сигнала.

15.3. Рассылка и генерация сигналов

Сигналы можно подавать вручную через `kill` и `raise()`:

- `kill(pid, signal)` — посылает сигнал другому процессу;

- `raise(signal)` — подаёт сигнал самому себе.

Поведение PID в `kill`:

- `> 1` — сигнал конкретному процессу;
- `0` — всем из текущей группы;
- `< 0` — всем из указанной группы по модулю;
- `-1` — всем процессам в системе (аккуратно!).

Сигналы — мощный, но технический инструмент. Их основное назначение — управление процессом: аварийное завершение, мягкое завершение, перезапуск, реакция на системные события.

16. Поток

17. Демон

Демон - процесс не имеющий связи с пользователем. Что то типа сервиса. Пример вебсервер. Демон пользовательская программа.

```
служебное хранилище всевозможных переменных
ls /var/run
// демон хранит pid номер в pid файле
*.pid /
```

17.1. Введение

18. Книжки

- корнев валединский 2023 задачник
- веревка достаточной длины
- роберт лав системное программирование
- Богачев основы параллельных вычислений
- даниель бабэ устройство ядра линукс